

When Edits Become History: Constrained Delegation for Long-Horizon Document Maintenance

Anonymous ACL submission

Abstract

Document agents carry each edited workspace into the next task. When an operation changes content outside the requested scope, later turns inherit that damage and can compound it. FULLREWRITE makes this failure possible even for local requests because the model must regenerate the entire editable workspace after every operation. We introduce HYBRIDPATCH, a constrained document write-back protocol. The model declares the files, locations, and operation type it intends to change. A deterministic executor applies only declared operations that pass validation and preserves all remaining content byte-for-byte. If validation and repair both fail, the current workspace is carried forward. The protocol supports local patches, bulk edits, block operations, and bounded file rewrites. We run paired comparisons on all 234 public DELEGATE-52 environments across model configurations, holding the model configuration, initial documents, task sequences, and evaluators fixed within each pair across ten edit-revert rounds. At round ten, the paired mean reconstruction advantages are 14.1 and 5.9 points; critical reconstruction drops by 31.4% and 10.5%, respectively. Constraining what each operation may write substantially reduces cumulative corruption and preserves document integrity over long editing sequences. Code is available at <https://anonymous.4open.science/r/HP-5E1C>.

1 Introduction

Long-horizon document maintenance links individual edits into a sequence of workspace states. After each task, the written workspace becomes the input to the next. When a write carries both the requested change and additional content changes, later tasks inherit the complete result. Early deviations therefore continue to participate in subsequent generations and gradually affect the integrity of the document.

DELEGATE-52 makes this process measurable (Laban et al., 2026). Each round trip contains a forward edit and a backward edit whose target is the initial state. The workspace produced by the backward edit becomes the input to the next forward edit. After every backward step, a domain-specific evaluator compares the current workspace with the initial document and computes a reconstruction score. The resulting trajectory measures content preservation across repeated editing. Prior analysis of DELEGATE-52 found that single-round reconstruction losses of at least 0.10 account for 80–98% of total degradation across evaluated models (Laban et al., 2026). We therefore report critical reconstruction drops alongside the endpoint score to measure the severe events that dominate the trajectory.

The DELEGATE-52 FULLREWRITE workflow implements every editing task as a complete workspace generation. The model produces the requested change while generating new versions of all editable files. The system commits that complete response and uses it as the input to the next task. At every step, the model therefore determines the next state of the entire workspace. A local edit receives workspace-wide write scope.

When a user assigns an edit, the write-back protocol determines how much of the workspace the resulting operation may replace. We call the design *constrained delegation* when this authority is limited to the files and locations declared for the operation; those targets define its write scope. HYBRIDPATCH implements constrained delegation with targeted operations. The model declares target files, edit footprint, and operation type, and a deterministic executor applies validated operations to the current workspace to construct the next state.

HYBRIDPATCH provides four write routes: local patches for targeted changes, bulk edits for repeated literal transformations, block operations for copying and distributing content, and bounded file

rewrites for file-level transformations. The model selects a route and generates its operation content; the executor resolves target locations, validates structure, and applies the transformation. Validation assigns each proposal one of three outcomes: direct commit, diagnosis followed by one repair generation, or the current workspace as the next state. Every outcome yields a complete workspace for the following task.

We compare HYBRIDPATCH with FULL-REWRITE on all 234 work environments in the public release of DELEGATE-52, spanning 48 professional domains, over ten consecutive edit-revert round trips. Within each paired model condition, both methods use the same initial workspaces, task sequences, generation settings, and domain-specific evaluators. Pooling the two equally sized model conditions, HYBRIDPATCH improves mean round-ten reconstruction by 14.6% relative to FULLREWRITE and reduces critical reconstruction drops by 22.2%.

Across the evaluation conditions, HYBRID-PATCH preserves more of the reconstructable workspace and incurs fewer severe drops that affect later rounds. Holding the model configuration fixed within each pair, changing the write-back protocol produces different long-horizon editing trajectories. The way model output is written into the workspace is therefore part of document-maintenance reliability.

We make three contributions:

- We formulate long-horizon document maintenance as a sequence of dependent workspace states and isolate write-back scope as a system-level control over which model-generated changes enter subsequent tasks.
- We introduce HYBRIDPATCH, which implements constrained delegation through structured operation declarations and deterministic execution so that each operation writes only to its named files and locations.
- Paired experiments over 234 environments and ten rounds show that constrained write-back improves long-horizon reconstruction and reduces critical reconstruction drops across model configurations.

2 Long-Horizon Document Editing

DELEGATE-52 reports a reconstruction score after each edit-revert round. The score measures how

closely the current workspace matches the initial workspace. To analyze how differences arise and persist, we examine how each operation transforms a current workspace W_t into a next workspace W_{t+1} .

An editing instruction usually requires changes to only part of a workspace, while the write-back process determines how much of the model output can replace the current state. When the effective write scope exceeds the content required by the task, generated differences in the additional region also enter W_{t+1} . Because W_{t+1} becomes the input to the following operation, those differences participate in later edits.

We use *task scope* for the content that must change to satisfy an instruction and *write scope* for the content that an operation is permitted to replace. We first formalize the state transitions in continuous editing and then analyze how the relation between these scopes affects subsequent workspaces.

2.1 Editing as State Transitions

Let W_0 be the initial workspace. Round i contains a forward instruction $x_i^{\rightarrow}$ and a backward instruction $x_i^{\leftarrow}$. An editor E applies them in sequence:

$$W_i^{\rightarrow} = E(W_{i-1}^{\leftarrow}, x_i^{\rightarrow}), \quad (1)$$

$$W_i^{\leftarrow} = E(W_i^{\rightarrow}, x_i^{\leftarrow}), \quad (2)$$

with $W_0^{\leftarrow} = W_0$. The forward instruction asks the editor to modify the current workspace. The backward instruction asks it to restore the affected content to its initial state. The resulting backward workspace $W_i^{\leftarrow}$ becomes the input to the next forward instruction.

After each backward edit, a domain-specific evaluator compares the current workspace with the initial workspace:

$$\text{RS}_i = \text{sim}_d(W_0, W_i^{\leftarrow}) \in [0, 1], \quad (3)$$

where sim_d implements the structural and content criteria for domain d . The sequence

$$W_0 \rightarrow W_1^{\rightarrow} \rightarrow W_1^{\leftarrow} \rightarrow W_2^{\rightarrow} \rightarrow W_2^{\leftarrow} \rightarrow \dots \quad (4)$$

records the workspace written after each operation. The output of round i is also the input to round $i + 1$, so a content change in one written workspace is included in subsequent editing rounds.

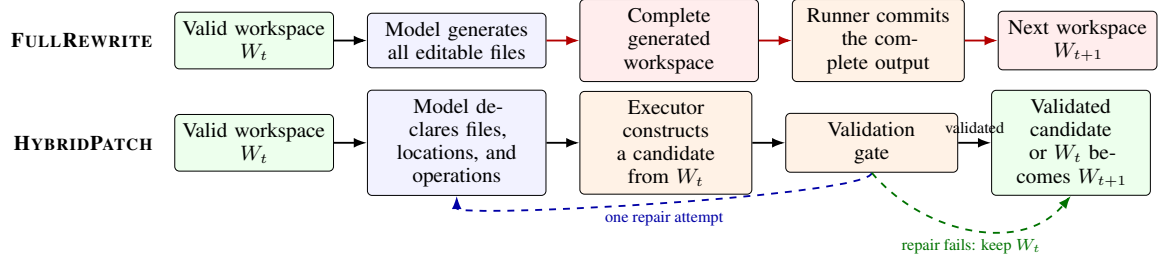


Figure 1: The protocols construct W_{t+1} from different model outputs. FULLREWRITE commits a completely generated editable workspace. HYBRIDPATCH applies declared operations to W_t and validates the candidate. A failed validation receives one repair attempt; after a failed repair, W_t becomes W_{t+1} .

2.2 Task Scope and Write Scope

Given a workspace W_t and instruction x_t , the model produces an edit proposal

$$P_t = M(W_t, x_t). \quad (5)$$

A write-back mechanism C then constructs the next workspace:

$$W_{t+1} = C(W_t, P_t). \quad (6)$$

The proposal describes the change the model intends to make. The write-back mechanism determines where the model output may replace the current state. A model may need to generate one local modification while the write-back mechanism permits replacement over a larger region.

Let T_t denote the task scope and A_t the write scope. A local instruction may have a task scope T_t consisting of one field, paragraph, or group of lines. File conversion and document splitting may require one or more complete files. When the system constrains the write scope to A_t , the executor applies the proposed change within A_t and carries the remaining content directly from the current workspace:

$$W_{t+1}[A_t] = W_t[A_t]. \quad (7)$$

Equation 7 defines how the next workspace is constructed: the model output determines new content within A_t , and the current workspace supplies content outside A_t . The model can still propose an incorrect edit within A_t , but the proposal remains bounded by that write scope.

Under FULLREWRITE, the model generates every editable file for each task. If $\mathcal{E}_t$ is the editable workspace, its effective write scope is

$$A_t^{\text{FR}} = \mathcal{E}_t. \quad (8)$$

Suppose a workspace contains B editable bytes and a task requires changes to b of them. A local edit commonly has $b \ll B$. FULLREWRITE still requires the model to generate all B bytes and commits the complete generated workspace as W_{t+1} .

A local task therefore regenerates the complete editable workspace, including files and content outside T_t . Changed field values, missing paragraphs, altered structure, and truncated files in the complete generation all become part of W_{t+1} and enter subsequent tasks with the workspace.

2.3 How Written Changes Enter Later Rounds

The previous sections distinguish task scope T_t from write scope A_t . This distinction matters over a long editing sequence because each written workspace becomes the input to the next operation.

Suppose step t changes a region R_t outside its task scope:

$$R_t \subseteq A_t \setminus T_t \quad \text{and} \quad W_{t+1}[R_t] \neq W_t[R_t]. \quad (9)$$

Once committed, the new content in R_t becomes part of W_{t+1} . The next model call receives that content with the rest of the workspace. Later operations may preserve it, overwrite it, or use it when generating related changes.

This state dependence establishes a path through which a change can affect later rounds. Its eventual effect depends on how long it persists, how document regions depend on one another, and which regions later tasks address. The reconstruction trajectory measures that effect across the editing sequence.

At the end of round i , the current workspace incorporates the results of every operation committed so far. The score

$$\text{RS}_i = \text{sim}_d(W_0, W_i^{\leftarrow}) \quad (10)$$

therefore measures their combined effect on the current state. The sequence $RS_1, \dots, RS_k$ records how document integrity changes across repeated write-back operations.

This analysis locates the control point in the construction of the next workspace. A write-back mechanism specifies where model output enters W_{t+1} and supplies the remaining content from W_t . HYBRIDPATCH implements this state construction with structured operation declarations and deterministic execution.

3 HybridPatch: Constrained Write-Back

HYBRIDPATCH places an explicit write-back process between a model proposal and the next workspace. The model interprets the instruction, selects an editing route, and generates the requested content. The executor determines the proposal's write scope, applies the declared operations, and constructs a candidate workspace. A validation gate then selects the workspace state supplied to the next operation.

The process has three stages. First, the model declares the target files, editing route, and operation content. Second, the executor applies those operations to the current workspace. Third, the validation gate determines the committed state. Figure 1 contrasts this process with complete workspace generation.

3.1 From an Instruction to Declared Operations

For a current workspace W_t and instruction x_t , the model receives the editable files and a deterministic file index. It returns a structured declaration containing an edit footprint, an editing route, target files, and route-specific operation content.

The edit footprint describes the scale of the task, such as specific locations, repeated content, indexed blocks, or complete files. The editing route determines how the executor interprets the content. The files and locations named by the operations define the write scope A_t .

For example, an instruction may request the same name change in two files. The model can select the bulk-edit route, name those files, and provide the replacement rule. The executor then derives A_t from the two declared files and applies the rule within that scope. The structured declaration converts the natural-language instruction into a state transformation that the executor can inspect

and apply.

3.2 Editing Routes

The editing route determines both the representation of a change and the information used to derive its write scope.

Local editing (`local_patch`). For changes at a small number of specific locations, each operation names a target file, a locating anchor, and replacement content. The executor uses these fields to update the corresponding locations.

Bulk editing (`bulk_patch`). For a literal transformation repeated across locations or files, the declaration lists the participating files together with the replacement or deletion rule.

Block editing (`dsl_rules`). For copying, moving, or distributing structured content, the declaration refers to source blocks and target positions through a deterministic block index.

Bounded file rewriting (`bounded_rewrite`). For conversion, splitting, or complete file reconstruction, the model supplies complete content for the declared target files. The executor updates those files with the generated content.

These routes cover editing tasks from local changes to complete file reconstruction. Every route uses named files and target locations to define A_t . Model-generated content enters that scope, and the executor copies the remaining workspace content from W_t .

3.3 Constructing a Candidate State

The executor uses W_t as the base for every route. For local and bulk editing, it first resolves all anchors and target locations against the same snapshot of W_t and checks their overlap relationships. It then applies accepted operations at the positions obtained from that snapshot, giving every operation a stable coordinate system.

For block editing, the executor identifies source content and target positions through the block index, then applies the declared copy or distribution rules. For bounded rewriting, the declared file set directly defines the write scope and the executor updates those files with the generated content.

All four routes follow the preservation condition introduced in Section 2:

$$W_{t+1}[\overline{A_t}] = W_t[\overline{A_t}]. \quad (11)$$

Content inside A_t comes from accepted model operations; content outside A_t comes directly from the current workspace. The executor checks this condition by comparing the input and candidate workspaces outside the recorded operation scope.

Task correctness additionally depends on the selected scope, the content generated within it, and completion of the instruction. The benchmark reconstruction score evaluates the recovered workspace; it does not separately score the quality of the forward edit.

3.4 Validating the Candidate State

The validation gate checks structural evidence from the instruction, declaration, and candidate workspace. It verifies that target files belong to the editable workspace, read-only files match their input state, operations satisfy their route schema, anchors and block identifiers resolve to unique locations, and required files are present. For a forward task that requires a change, the gate also checks that the candidate workspace contains an effective modification.

A candidate that passes these checks becomes W_{t+1} . A rejected candidate produces a diagnostic record such as a missing target, ambiguous anchor, operation conflict, or missing required structure. The model then receives one repair opportunity containing W_t , the original instruction, and the diagnostic record.

The repair proposal is executed against W_t and passes through the same validation gate. A validated repair becomes W_{t+1} . A rejected repair carries the current workspace into the next step:

$$W_{t+1} = W_t. \quad (12)$$

Local and bulk declarations may contain several independent operations. A restricted partial-acceptance rule commits the accepted subset when the declaration contains both accepted and rejected operations and the resulting candidate satisfies every remaining validation condition. The executor records the rejected operations with the committed state.

Operation declarations determine write scope, the executor constructs a candidate from the current state, and the validation gate selects the state supplied to the next operation. Together, these stages give every write-back step an explicit set of files and locations that it may change.

4 Experimental Setup

We use paired experiments to measure how the write-back protocol affects a workspace maintained over repeated edits. Within each pair, HYBRIDPATCH and FULLREWRITE receive the same initial workspace, task sequence, and model configuration. Each method then continues from the state produced by its own preceding operation.

4.1 Benchmark and Task Sequences

DELEGATE-52 contains reversible editing tasks for professional document formats (Laban et al., 2026). The public dataset provides 234 work environments across 48 domains, drawn from the benchmark’s full collection of 52 domains. Each environment contains an initial workspace, reversible edit pairs, distractor context, and a domain-specific evaluator. Every reported experimental condition covers all 234 public environments.

For each environment, the experiment executes ten forward–backward task pairs. A forward instruction modifies the current workspace, and the paired backward instruction requests restoration of the affected content. The backward output is passed directly to the next forward instruction. Each method therefore performs 20 operations per environment and produces ten reconstruction measurements.

4.2 Paired Comparison

The compared systems differ in how they commit model-generated edits. FULLREWRITE follows the complete-workspace generation protocol in the benchmark runner. HYBRIDPATCH uses the hybridpatch protocol described in Section 3. The reported conditions use MiniMax-M3 (Lai et al., 2026) and DeepSeek-V4-Flash-0731 (DeepSeek-AI, 2026), with all 234 environments evaluated under each configuration. Each paired comparison fixes the initial workspaces, task sequences, model configuration, generation settings, transport, and domain evaluators while varying the write-back protocol. The two methods maintain independent workspace states and pass each committed state to the next task. HYBRIDPATCH permits one repair generation after a diagnosed protocol or execution failure; FULLREWRITE performs one complete-workspace generation for each task.

4.3 Measures

The underlying reconstruction score produced by each domain-specific evaluator lies in $[0, 1]$. Following DELEGATE-52, we multiply these scores by 100 in all tables and reported numerical comparisons. Reported score differences and confidence intervals are expressed in points, while percentages are reserved for event rates and relative changes. RT_i denotes the reconstruction score after the i th edit–revert round trip and corresponds to $RS@2i$ in DELEGATE-52.

Our primary outcome is the reconstruction score after the tenth backward edit, RS_{10} . For each experimental condition, we compute the mean paired difference across the 234 environments and an environment-level paired bootstrap 95% confidence interval. We report these differences and confidence intervals on the 0–100 scale in points. Round-level means from RS_1 through RS_{10} describe how the difference develops over the editing sequence. Statistical inference uses one final endpoint per environment within each condition.

We also measure large losses between consecutive backward states. A *critical reconstruction drop* occurs when

$$RS_{i-1} - RS_i \geq 0.10 \quad (13)$$

or when a positive score falls to zero. The threshold of 0.10 on the underlying scale is equivalent to a drop of 10 points on the reported scale. With nine adjacent transitions in each of 234 environments, each method has 2,106 opportunities for a critical drop.

A rejected proposal can carry the input state forward and leave a forward instruction incomplete. We therefore include a symmetric no-op sensitivity analysis. If a method produces a forward workspace that is byte-identical to its input over the editable files, the backward reconstruction score for that round is set to zero. All 234 environments remain in the analysis. This analysis measures the contribution of unchanged forward states to the endpoint difference.

5 Results

After one edit–revert pair, the reconstruction scores produced by the two write-back protocols remain close. As committed workspaces feed into later tasks, the gaps widen: **the mean differences are 2.3 and 2.7 at round one, and 14.1 and 5.9 at round ten.**

The round-ten paired differences are positive in both conditions, with 95% confidence intervals of $[9.4, 19.0]$ and $[1.6, 10.5]$.

The two protocols exhibit different degradation trajectories across the editing sequence. From RT_1 to RT_{10} , mean reconstruction with HYBRIDPATCH falls by 16.7 and 16.3 in the two conditions, whereas mean reconstruction with FULLREWRITE falls by 28.5 and 19.5. The smaller declines under HYBRIDPATCH indicate that constrained write-back retains more of the reconstructable workspace over repeated edit–revert operations.

HYBRIDPATCH carries the current workspace forward when a proposal fails validation, which can leave a forward task incomplete. Byte-identical forward workspaces occur in 2.95% and 3.63% of HYBRIDPATCH operations, compared with 0.38% and 0.09% for FULLREWRITE. Assigning a zero backward score to these rounds yields round-ten mean differences of **11.7** and **3.7**, preserving 83% and 62% of the original gaps. The mean reconstruction advantage remains positive after incomplete forward tasks receive this penalty.

Critical reconstruction drops capture large single-round losses along the editing sequence. Under MiniMax-M3, HYBRIDPATCH has 116 critical drops across 2,106 adjacent transitions, compared with 169 for FULLREWRITE, corresponding to 53 fewer drops and a relative reduction of **31.4%**. Under DeepSeek-V4-Flash-0731, HYBRIDPATCH has 119 critical drops compared with 133 for FULLREWRITE, corresponding to 14 fewer drops and a relative reduction of **10.5%**. Because each backward workspace becomes the input to the next task, a critical drop lowers the starting point for subsequent editing. Across both model configurations, HYBRIDPATCH is associated with fewer critical drops and ends RT_{10} with a higher reconstruction score.

6 Discussion

This study exposes a mismatch of authority in document agents. A user typically delegates an edit to a specific part of a document, yet the system may grant the model write access to the entire workspace. FullRewrite turns a local request into a workspace-wide write, making every difference in the model output persistent. Continuous maintenance magnifies the consequences because each written state carries the next task. Long-horizon reliability therefore depends on the model’s abil-

Table 1: Mean reconstruction score from RT1 to RT10. Each round trip contains one forward edit and one backward edit, corresponding to two model interactions. All results are computed over the same 234 public DELEGATE-52 environments.

Model	Method	RT1	RT2	RT3	RT4	RT5	RT6	RT7	RT8	RT9	RT10
 MiniMax-M3	HYBRIDPATCH	94.8	90.2	88.2	86.6	85.6	82.0	79.6	78.9	78.1	78.1
	FULLREWRITE	92.5	85.3	79.1	76.4	73.9	70.0	67.2	66.2	63.7	64.0
 DeepSeek-V4-Flash-0731	HYBRIDPATCH	95.4	88.7	88.0	85.9	84.8	83.6	82.4	81.2	80.2	79.1
	FULLREWRITE	92.7	86.6	84.2	81.6	79.4	76.4	74.8	73.7	73.1	73.2

ity to complete the current edit and on how much document history the system allows that edit to change.

The effect appears under both model configurations and varies in magnitude. At RT10, reconstruction gains are 14.1 points for MiniMax-M3 and 5.9 points for DeepSeek-V4-Flash-0731; critical drops fall by 31.4% and 10.5%, respectively. The common direction shows that write-back rules affect both models, while the different magnitudes show that the base model shapes the resulting trajectory.

This observation changes what document-editing competence means. Reliable maintenance has two concurrent obligations: producing the requested content and carrying untouched content intact into the next round. Single-turn task success measures the requested change. A persistent workspace adds the requirement to preserve everything the task leaves alone. Once later tasks inherit earlier outputs, preservation becomes as important as generation. An output can satisfy the current instruction and still leave subsequent tasks with a degraded starting point. Evaluation for persistent documents should therefore measure the target edit, preservation of untouched content, and the final state produced by an extended editing sequence.

The system architecture provides another source of reliability. Models can generate the content that must change, while deterministic mechanisms carry existing state forward. HYBRIDPATCH assigns these duties according to the declared write scope: the model proposes new content for named files and locations, and the executor inherits the rest from the current workspace. This division retains the model’s flexibility for open-ended editing and removes unchanged state from its generative burden. Model capability determines proposal quality; write-back rules determine which parts of a proposal enter the document’s history. Together, they shape the system’s long-horizon behavior.

Constrained write-back also gives failure a clear system meaning. Validated operations enter the workspace within their declared scopes. When the initial candidate and its repair both fail validation, the current state remains intact and the task remains visibly unfinished. The failure can then be inspected or escalated while its effect remains local to the current operation. In professional document maintenance, a recoverable task failure is easier to handle than a silent content change that has already entered the document history. Deployment evaluations should therefore record task completion, state preservation, and failure handling separately, so the system can decide when to continue, regenerate, or request confirmation.

More broadly, the write authority granted to each action shapes the reliability of a long-horizon agent. Better prompts, self-correction, and stronger base models can improve proposed edits. Explicit write scopes limit how much of an uncertain output becomes history. Our results support writing document edits through operations that declare their files and locations. The model proposes the content and its write scope; the executor validates the operation and applies it within that scope. Constrained delegation connects model generation to deterministic control over persistent change and provides a path to more reliable maintenance alongside model improvements.

7 Related Work

HYBRIDPATCH derives write scope from the files and locations named in an edit proposal. Text-editing research has developed explicit representations of operation type and target location. LaserTagger reconstructs a target text through keep, delete, and phrase-insertion operations, while FELIX uses tagging and insertion to select, reorder, and augment input tokens (Malmi et al., 2019; Mallinson et al., 2020). IteraTeR and subsequent

location-aware revision work incorporate edit intent, granularity, and location into iterative revision (Du et al., 2022; Kim et al., 2022). PEER, CoEdit, and EditEval study revision driven by natural-language instructions (Schick et al., 2023; Raheja et al., 2023; Dwivedi-Yu et al., 2024). This line of work represents edit content, operation type, and target location as model outputs. HYBRIDPATCH turns those fields into executor inputs. The operation type selects an editing route, and the named files and locations define the write scope. An edit proposal thus describes both the requested content and where that content may be written.

Long-document editing shows why a local change may require information from distant parts of a document. LEDGER uses a dependency graph to identify affected document elements and retrieve their structural and semantic context (Wang et al., 2026). Its design separates read scope from modification scope: dependencies determine what the model needs to inspect, while the affected elements determine where the edit is applied. The same separation appears in HYBRIDPATCH as proposal generation and write-back. The model forms a proposal from workspace context, and the executor constructs the next state from the declared targets.

Applying a declared scope requires programmatic validation of the model proposal. PICARD rejects inadmissible tokens through incremental parsing, and grammar-constrained decoding restricts outputs with a formal grammar (Scholak et al., 2021; Geng et al., 2023). JSON Patch represents state transformations through operations and paths and specifies failure semantics for operations that cannot be applied (Bryan and Nottingham, 2013). These mechanisms allow a program to inspect generated structures before applying them. HYBRIDPATCH separates proposal generation from deterministic execution and validates proposals against the current workspace. The executor checks target files, editing routes, and the unique resolution of anchors or block identifiers before applying accepted operations. The structured proposal gives the executor a computable write scope and checkable operation conditions.

Once a structured proposal is executed, the editing protocol becomes an action interface between the agent and its environment. ReAct connects model reasoning to external actions, and SWE-agent shows that the action space and agent-computer interface affect the behavior and performance of the same underlying model (Yao et al.,

2023; Yang et al., 2024). ToolEmu studies risks introduced by tool actions and their side effects (Ruan et al., 2024). These results make action semantics part of the agent system. In document editing, those semantics are defined by the write-back protocol. FULLREWRITE constructs the next state from a complete workspace generated by the model; HYBRIDPATCH constructs it from declared operations and the current workspace. The two interfaces grant different persistent write scopes, so generation errors enter subsequent workspaces under different state-transition rules.

Agent benchmarks provide ways to measure these environmental consequences. SWE-bench evaluates repository modifications with executable tests (Jimenez et al., 2024). AgentBench and WebArena evaluate outcomes in multi-step interactive environments (Liu et al., 2024; Zhou et al., 2024); τ -bench checks tool-mediated outcomes against domain policies (Yao et al., 2025); and AppWorld evaluates task completion together with changes to application state (Trivedi et al., 2024). In DELEGATE-52, the workspace produced by each edit is passed to the next task, allowing the state created by a write-back action to propagate through the sequence (Laban et al., 2026). This chained state exposes the long-term effects of the write-back protocol. Our paired experiments compare the two protocols under identical task sequences and domain evaluators, using reconstruction trajectories to measure their effects on subsequent workspace integrity.

8 Conclusion

In long-horizon document maintenance, every written workspace becomes the input to the next task. A document agent must complete the current edit and carry content untouched by that task into later work. We compare two ways of doing so. FULLREWRITE asks the model to generate the complete next workspace. HYBRIDPATCH implements constrained delegation by asking the model to name the files, locations, and operations to change, then applying validated operations to the current workspace. All remaining content comes directly from the preceding state.

Across the evaluation conditions, HYBRIDPATCH achieves higher reconstruction after ten edit-revert rounds, with paired differences of 14.1 and 5.9 points. Critical reconstruction drops fall from 169 to 116 and from 133 to 119. With the

model configuration, initial workspace, and task sequence held fixed within each pair, changing the write-back protocol reduces the accumulation of errors across consecutive tasks.

HYBRIDPATCH assigns new content and existing content to different parts of the system. The model generates content for the accepted operations, and the executor copies everything outside the declared write scope from the preceding workspace. A local task therefore commits only the accepted operations in its proposal, and the next task continues from that locally updated state. Long-horizon document maintenance depends on accurate generation and on a write-back process that limits each commit to its declared write scope.

9 Limitations

Our evaluation covers long-horizon reconstruction and workspace preservation under the public DELEGATE-52 protocol. The benchmark uses domain-specific evaluators to measure workspace reconstruction after the model modifies and then restores the relevant content. Evaluating the quality of individual editing tasks requires reference targets, while sequences of irreversible edits require a different task design. Our results characterize reconstruction and preservation under the released protocol. Broader claims about document-editing quality require direct evaluation in these settings.

We evaluate all 234 publicly available work environments spanning 48 domains under two model configurations. Each full paired evaluation requires thousands of model calls, so each environment contributes one paired trajectory. This design provides full coverage of the public release. Estimating within-environment run-to-run variation requires repeated trajectories, and additional model configurations would extend the analysis of result stability.

HYBRIDPATCH relies on the model to select an appropriate editing route and identify the files and locations to modify. The validation gate checks the structural validity of the declared operations and their applicability to the current workspace. The model remains responsible for matching those targets to the scope intended by the natural-language instruction. An independent intent-to-scope verifier would extend this part of the protocol.

References

Paul Bryan and Mark Nottingham. 2013. [JavaScript Object Notation \(JSON\) Patch](#). Technical Report

RFC 6902, Internet Engineering Task Force.

DeepSeek-AI. 2026. [DeepSeek-V4: Towards highly efficient million-token context intelligence](#). *Preprint*, arXiv:2606.19348.

Wanyu Du, Vipul Raheja, Dhruv Kumar, Zae Myung Kim, Melissa Lopez, and Dongyeop Kang. 2022. [Understanding iterative revision from human-written text](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3573–3590. Association for Computational Linguistics.

Jane Dwivedi-Yu, Timo Schick, Zhengbao Jiang, Maria Lomeli, Patrick Lewis, Gautier Izacard, Edouard Grave, Sebastian Riedel, and Fabio Petroni. 2024. [EditEval: An instruction-based benchmark for text improvements](#). In *Proceedings of the 28th Conference on Computational Natural Language Learning*, pages 69–83. Association for Computational Linguistics.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. [Grammar-constrained decoding for structured NLP tasks without finetuning](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952. Association for Computational Linguistics.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. [SWE-bench: Can language models resolve real-world GitHub issues?](#) In *International Conference on Learning Representations*.

Zae Myung Kim, Wanyu Du, Vipul Raheja, Dhruv Kumar, and Dongyeop Kang. 2022. [Improving iterative text revision by learning where to edit from other revision tasks](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 9986–9999. Association for Computational Linguistics.

Philippe Laban, Tobias Schnabel, and Jennifer Neville. 2026. [LLMs corrupt your documents when you delegate](#). *arXiv preprint arXiv:2604.15597*.

Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. 2026. [MiniMax Sparse Attention](#). *Preprint*, arXiv:2606.13392.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. [AgentBench: Evaluating LLMs as agents](#). In *International Conference on Learning Representations*.

Jonathan Mallinson, Aliaksei Severyn, Eric Malmi, and Guillermo Garrido. 2020. [FELIX: Flexible text editing through tagging and insertion](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1244–1255. Association for Computational Linguistics.

821	Eric Malmi, Sebastian Krause, Sascha Rothe, Daniil	agent-user interaction in real-world domains. In <i>In-</i>	878
822	Mirylenka, and Aliaksei Severyn. 2019. Encode, tag,	<i>ternational Conference on Learning Representations.</i>	879
823	realize: High-precision text editing. In <i>Proceedings</i>		
824	<i>of the 2019 Conference on Empirical Methods in Nat-</i>	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak	880
825	<i>ural Language Processing and the 9th International</i>	Shafraan, Karthik Narasimhan, and Yuan Cao. 2023.	881
826	<i>Joint Conference on Natural Language Processing,</i>	ReAct: Synergizing reasoning and acting in language	882
827	pages 5054–5065. Association for Computational	models. In <i>International Conference on Learning</i>	883
828	Linguistics.	<i>Representations.</i>	884
829	Vipul Raheja, Dhruv Kumar, Ryan Koo, and Dongyeop	Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou,	885
830	Kang. 2023. CoEdit: Text editing by task-specific	Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue	886
831	instruction tuning. In <i>Findings of the Association</i>	Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Gra-	887
832	<i>for Computational Linguistics: EMNLP 2023,</i> pages	ham Neubig. 2024. WebArena: A realistic web envi-	888
833	5274–5291. Association for Computational Linguis-	ronment for building autonomous agents. In <i>Inter-</i>	889
834	tics.	<i>national Conference on Learning Representations.</i>	890
835	Yangjun Ruan, Honghua Dong, Andrew Wang, Sil-		
836	viu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois,		
837	Chris Maddison, and Tatsunori Hashimoto. 2024.		
838	Identifying the risks of LM agents with an LM-		
839	emulated sandbox. In <i>International Conference on</i>		
840	<i>Learning Representations.</i>		
841	Timo Schick, Jane A. Yu, Zhengbao Jiang, Fabio		
842	Petroni, Patrick Lewis, Gautier Izacard, Qingfei You,		
843	Christoforos Nalmpantis, Edouard Grave, and Sebas-		
844	tian Riedel. 2023. PEER: A collaborative language		
845	model. In <i>International Conference on Learning</i>		
846	<i>Representations.</i>		
847	Torsten Scholak, Nathan Schucher, and Dzmitry Bah-		
848	danau. 2021. PICARD: Parsing incrementally for		
849	constrained auto-regressive decoding from language		
850	models. In <i>Proceedings of the 2021 Conference on</i>		
851	<i>Empirical Methods in Natural Language Processing,</i>		
852	pages 9895–9901. Association for Computational		
853	Linguistics.		
854	Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin		
855	Manku, Vinty Dong, Edward Li, Shashank Gupta,		
856	Ashish Sabharwal, and Niranjan Balasubramanian.		
857	2024. AppWorld: A controllable world of apps and		
858	people for benchmarking interactive coding agents.		
859	In <i>Proceedings of the 62nd Annual Meeting of the</i>		
860	<i>Association for Computational Linguistics (Volume</i>		
861	<i>1: Long Papers),</i> pages 16022–16076. Association		
862	for Computational Linguistics.		
863	Hang Wang, Utkarsh Garg, Reza Davari, Huitian Jiao,		
864	Hao Cheng, Baolin Peng, Si-Qing Chen, and Tao		
865	Ge. 2026. LEDGER: Scaling agentic document edit-		
866	ing with dependency-aware graph retrieval. In <i>Find-</i>		
867	<i>ings of the Association for Computational Linguis-</i>		
868	<i>tics: ACL 2026,</i> pages 10614–10644. Association for		
869	Computational Linguistics.		
870	John Yang, Carlos E. Jimenez, Alexander Wettig, Kil-		
871	ian Lieret, Shunyu Yao, Karthik Narasimhan, and		
872	Ofir Press. 2024. SWE-agent: Agent-computer in-		
873	terfaces enable automated software engineering. In		
874	<i>Advances in Neural Information Processing Systems,</i>		
875	volume 37.		
876	Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik		
877	Narasimhan. 2025. τ-bench: A benchmark for tool-		